

1.6 From Recognition to Evaluation: where the group report sits, what it already does, and what is still open

► LEARNING OBJECTIVE

Rather than evaluating speech recognition purely as text transcription, this section examines how an edge voice processing system scores a learner's read-aloud utterance. We place the research group's internal technical report (BAO_CA0_492026.pdf, 32 slides across four September 2026 meetings: 04/09, 11/09, 18/09, 23/09) inside the end-to-end evaluation flow, reconstruct the mathematical and geometric contract established across the slides, articulate what that contract already accomplishes, systematically analyze four independent open cases, formulate architectural candidates across uniform evaluation axes, and prepare an actionable decision table for the team without pre-judging the group's choice.

1.6.0 Where the Report Sits in Read-Aloud Speech Evaluation

Automated speech scoring in educational technology is fundamentally distinct from automatic speech recognition (ASR). An ASR engine maps continuous acoustic waveforms into discrete orthographic text tokens, maximizing statistical transcription accuracy. In contrast, an automated pronunciation assessment system must evaluate a student who is reading a known prompt aloud. In this setting, the orthographic text is already given; the engineering objective is to determine how well, how accurately, and how naturally the learner articulated the target text.

Scoring a spoken read-aloud task requires five distinct processing stages:

1. **Stage 1: Audio Capture and Acoustic Conditioning.** Digitizing the raw microphone signal, removing direct-current (DC) offset, normalizing energy across varying mouth-to-microphone distances, and executing Voice Activity Detection (VAD) to trim leading and trailing non-speech silences.
2. **Stage 2: Word Edge Localization (Forced Alignment).** Segmenting the continuous audio stream into discrete word intervals by computing acoustic boundaries $[t_{\text{start}}, t_{\text{end}}]$ for each orthographic token in the prompt.
3. **Stage 3: Phonetic Correctness Verification (Acoustic Quality Scoring).** Evaluating whether the spectral realization of each segment matches the target phonemes (e.g., verifying formant trajectories F_1, F_2, F_3 , computing Goodness of Pronunciation [GOP] log-likelihood ratios, or detecting phoneme substitutions, deletions, and insertions).
4. **Stage 4: Melodic and Prosodic Comparison Against Reference Audio.** Extracting prosodic contours—primarily fundamental frequency (F_0), syllabic duration, and energy dynamics—and comparing their trajectory against an exemplar spoken reference.
5. **Stage 5: Labeling and Score Calibration.** Aggregating acoustic and prosodic measurements into pedagogical rubrics, mapping continuous numerical scores to discrete CEFR levels or word-level pass/fail color codes, and providing actionable feedback to the learner.

Placement of the Group Report. The 32 slides of BAO_CA0_492026.pdf occupy **Stage 4** (Melodic and prosodic comparison via Pearson pitch correlation) and provide an initial heuristic toward **Stage 5** (word-level pass/fail labeling via sliding window thresholding). The report does not implement Stages 1, 2, or 3, but rather makes specific operational assumptions about them:

- **Stage 1 is assumed already resolved:** On Slide 25, the pipeline executes upon a pre-processed file explicitly named `sarah_trimmed.wav`. The silence detection, DC removal, and boundary cropping are assumed to be performed offline before the pipeline begins.



Figure 1: **The five-stage read-aloud evaluation flow and the placement of the group report.** The 32 slides of `BAO_CAO_492026.pdf` occupy Stage 4 (Pearson pitch correlation) and an initial heuristic rule in Stage 5 ($r < 0.3$ window thresholding), while treating Stages 1, 2, and 3 as already satisfied or unneeded.

- **Stage 2 is assumed unneeded:** Rather than deploying a forced alignment acoustic model to identify exact temporal word boundaries $[t_{\text{start}}, t_{\text{end}}]$, the report constructs an analytical sliding window model (Slides 19–23) that slices the sentence into $K + 1$ overlapping geometric segments based solely on word count K and total sentence length N .
- **Stage 3 is assumed to follow from pitch correlation:** The report treats the correlation of fundamental frequency (F_0) as an indicator of whether the user read correctly (“checking whether the user reads like the model”, Slide 2), assuming that a learner matching the intonation contour also pronounced the underlying phonemes accurately.

Sarah is an Exemplar Ruler, Not Linguistic Ground Truth. Throughout the four meeting reports, the target reference is denoted as “Sarah” (Slide 5, 12, 25, 32). In the group’s engineering baseline, Sarah is a specific native recording used as a comparative ruler. It is essential to recognize that Sarah represents an exemplar reference, not absolute linguistic ground truth. In human speech, two native speakers pronouncing the exact same sentence correctly will exhibit different baseline pitch registers (e.g., an adult male voice with $F_0 \in [85, 155]$ Hz versus an adult female voice with $F_0 \in [165, 255]$ Hz), distinct dynamic pitch excursions, and natural tempo fluctuations across syllables.

By selecting Sarah’s audio as the sole comparative target, the pipeline adopts an engineering simplification: evaluating a student’s prosodic contour against one fixed performance. Understanding where the report sits in this broader five-stage hierarchy allows a new member of the research group to appreciate what the slides deliberately chose to isolate, what they built, and what remains open for team decision.

1.6.1 The Evaluation Contract as the Slides Wrote It

Across four formal group presentations in September 2026, the research team established an explicit algorithmic and geometric contract to score spoken utterances. Every equation, worked numeric trace, and threshold in this section is extracted directly from the slide deck `BAO_CAO_492026.pdf`.

Meeting 04/09/2026: Intonation Correlation via Pearson r (Slides 1–9)

The initial presentation defines the core problem statement: “*Checking whether the graphs of two pitch contours have the same trend?*” (Checking whether two pitch contours follow the same trend, Slide 1), proposing Pearson correlation as the comparative engine: “*Pearson Correlation Coefficient: checking whether the user reads like the model*” (Slide 2).

On Slide 5, the report formulates the mathematical definition:

$$r = \frac{\text{Cov}(X, Y)}{\sigma_X \sigma_Y} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}} \quad (1)$$

The slide explicitly assigns variable definitions:

- n : “*total duration of a word*” (the total duration or discrete sample point count representing the word).

- $X = \{x_1, x_2, \dots, x_n\}$: “pitch array of a word from User” (discrete pitch array of the learner).
- $Y = \{y_1, y_2, \dots, y_n\}$: “pitch array of a word from Sarah” (discrete pitch array of the reference recording).
- $\bar{x}, \bar{y}$: “arithmetic mean pitch” (the arithmetic mean pitch of User and Sarah).

Mathematical Rationale from the Slides. The deductive properties of Pearson correlation are articulated across Slides 6–8:

1. **Directional Trend Matching (Numerator):** The product $(x_i - \bar{x})(y_i - \bar{y})$ tests co-directional inflection. If the learner’s pitch rises when Sarah’s pitch rises (both above their respective means), or falls when Sarah’s falls, the product is positive (“*Same trend*”, Slide 7). Conversely, if the learner’s pitch rises while Sarah’s falls, the product is negative (“*Opposite trend*”, Slide 7).
2. **Pitch Register Invariance (Denominator):** The normalization by $\sigma_X \sigma_Y$ rescales the covariance strictly into the interval $[-1, +1]$ (Slide 8). This eliminates the effect of absolute pitch magnitude (“*removes differences in pitch magnitude*”, Slide 6), enabling comparison between a low-pitched male voice and a high-pitched female voice, and normalizes dynamic intonation excursion range (“*cancels out differences in pitch excursion*”, Slide 8).

Slide 9 provides an external bibliographic reference for the statistical derivation: <https://www.statisticssolution.com/free-resources/directory-of-statistical-analyses/pearsons-correlation-coefficient/>.

Meeting 11/09/2026: Resampling Preprocessing and the Sliding Window Model (Slides 10–23)

The presentation of 11/09/2026 addresses two central requirements: enforcing identical array lengths and localizing evaluation across multi-word sentences under the title “*SLIDING WINDOW PEARSON*” (Sliding Window Pearson, Slide 10).

Part 1: Linear Resampling Preprocessing (Slides 11–18). Equation (1) strictly requires arrays X and Y to contain an identical number of points n . Because human speakers read at different speaking rates, the raw pitch arrays inevitably diverge in length. Slide 11 defines the preprocessing contract: “*Resize length so user equals Sarah (Pearson requires equal array lengths: using linear interpolation)*”.

To demonstrate this preprocessing step, Slides 12–18 present a concrete worked example (“VD1”):

- Input User array: User = [40, 90, 100, 60, 120, 145] (6 points).
- Target Sarah array: Sarah = [30, 100, 60, 80] (4 points).

The engineering goal is to resample the 6-point User array down to 4 points to match Sarah. Mapping both sequences onto a normalized continuous coordinate $x \in [0, 1]$:

- The 6 User points are located at uniform coordinates $x \in \{0.0, 0.2, 0.4, 0.6, 0.8, 1.0\}$.
- The 4 Sarah target points are located at $x \in \{0.0, 0.33, 0.67, 1.0\}$.

Slides 14–17 print the exact arithmetic executed for each target index:

1. **At $x = 0.0$ (Slide 14):** User[0] = 40.

2. **At $x = 0.33$ (Slide 15):** The target coordinate 0.33 lies in the interval $[0.2, 0.4]$ between $\text{User}(0.2) = 90$ and $\text{User}(0.4) = 100$. The fractional distance across the sub-interval is $\alpha = \frac{0.33-0.20}{0.40-0.20} = \frac{0.13}{0.20} = 0.65$. The interpolated value is:

$$\text{User}[1] = 90 + 0.65 \times (100 - 90) = 90 + 6.5 = 96.5 \quad (2)$$

3. **At $x = 0.67$ (Slide 16):** The target coordinate 0.67 lies in the interval $[0.6, 0.8]$ between $\text{User}(0.6) = 60$ and $\text{User}(0.8) = 120$. The fractional distance across the sub-interval is $\alpha = \frac{0.67-0.60}{0.80-0.60} = \frac{0.07}{0.20} = 0.35$. The interpolated value is:

$$\text{User}[2] = 60 + 0.35 \times (120 - 60) = 60 + 21.0 = 81.0 \quad (3)$$

4. **At $x = 1.0$ (Slide 17):** The terminal sample is assigned directly: $\text{User}[4] = 145$ (labeled as $\text{User}[4]$ on Slide 17, corresponding to the final target sample).

On Slide 18, the resulting resampled array is printed in full:

$$\text{User}_{\text{resized}} = [40, 96.5, 81, 145] \quad (4)$$

Convention Note on VD1: The slide states the interpolation factors 0.65 and 0.35 directly. While the exact node-endpoint convention in Python code (e.g., `np.interp` endpoints versus discrete coordinate rounding) is underspecified in the slide text, the printed arithmetic corresponds precisely to linear interpolation at normalized coordinates $x = 0.33$ and $x = 0.67$. Slide 18 concludes with the candid operational note: “*Asking the chatbot to explain makes it easier to understand*”.

Part 2: The Sliding Window Model (Slides 19–23). Rather than computing a single global Pearson correlation across the entire sentence, the group introduced an analytical sliding window model (Slides 19–22) to localize prosodic evaluation. The formulation establishes four parameters:

- N : total length of the sentence (in samples or frames).
- K : number of orthographic words in the sentence prompt.
- W : analysis window size (in samples or frames).
- S : window shift step (stride).

On Slides 20–22, the group adopts the specific parameterization: “*Assuming $S = W/2$, we slide $K + 1$ times (instructor proposal)*”. Setting the stride to a 50% overlap ($S = W/2$), covering the full sentence length N in exactly $K + 1$ overlapping windows yields the geometric sizing identity:

$$N = W + K \times S \iff N = W + K \times \left(\frac{W}{2}\right) = W \left(1 + \frac{K}{2}\right) = W \left(\frac{K+2}{2}\right) \quad (5)$$

Solving algebraically for window width W yields:

$$W = \frac{2N}{K+2}, \quad S = \frac{W}{2} = \frac{N}{K+2} \quad (6)$$

The Group Condemnation Rule. On Slide 23, the group couples this window geometry with an explicit scoring decision rule:

“If any window has $r < 0.3$, then all words falling on that window are marked incorrect, even if they also fall on another window with $r \geq 0.3$.”

Under this rule, a threshold of $r = 0.3$ serves as the boundary between acceptable intonation and error. Crucially, the rule is asymmetrical and condemnatory: if a word spans across multiple overlapping windows, any single window dropping below 0.3 permanently stains and fails all overlapping words, regardless of high correlation in neighboring windows.

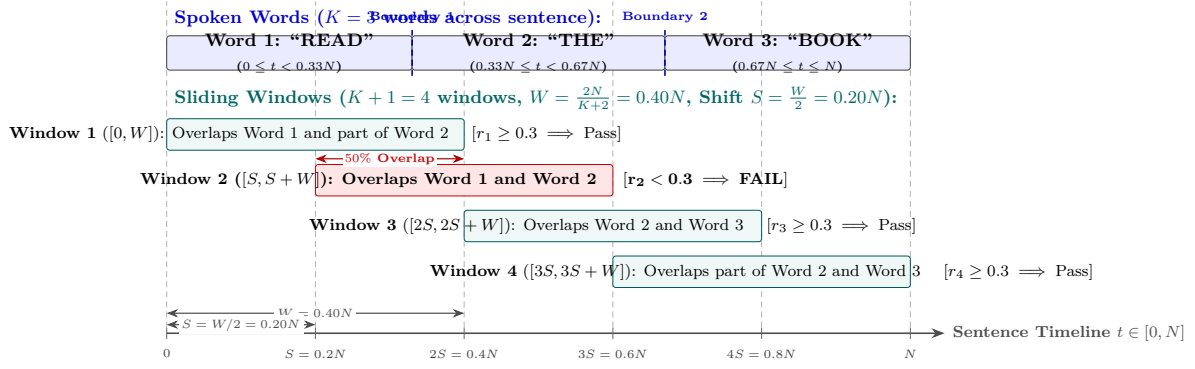


Figure 2: **Figure 1.6a: The Sliding Window Contract and the Group Condemnation Rule.** Visualizing the group’s geometric model for a $K = 3$ word sentence (“READ THE BOOK”) spanning N frames. With 50% overlap ($S = W/2$), the pipeline constructs $K + 1 = 4$ windows of width $W = 2N/(K + 2) = 0.40N$ and stride $S = 0.20N$. If Window 2 exhibits an intonation mismatch ($r_2 < 0.3$), the group condemnation rule stains both Word 1 and Word 2 as incorrect, even though Word 1 also overlaps Window 1 ($r_1 \geq 0.3$).

Meeting 18/09/2026: Pitch Grid Regularity and Praat Centering Verification (Slides 24–30)

The third meeting investigates the temporal regularity of the extracted pitch grid under the title “*PITCH ARRAY EVENLY DISTRIBUTED AT 0.01s: To ensure that linear interpolation can be applied soundly*” (Slide 24).

Slide 25 formulates an empirical question based on a real test file:

“Proving pitch values recorded by Praat are spaced at 0.01s (One might wonder why, with 0.01s spacing and a 1.450667s audio duration, Praat extracts 142 pitch points rather than 145, starting at 0.0203s instead of 0.01s)...”

When parsing `sarah_trimmed.wav` (duration $T_{\text{audio}} = 1.450667$ s), Praat extracts exactly 142 pitch points, with timestamps starting at $t_1 = 0.02033333$ s and continuing at regular 0.01 s increments (0.03033333, 0.04033333, ...).

Checking the Praat Manual. Slide 26 cites the Praat documentation (https://www.fon.hum.uva.nl/praat/manual/Intro_4_2__Configuring_the_pitch_contour.html) and Parselmouth. Opening the Praat manual reveals the exact sizing formulas:

“If the pitch floor is 50 Hz, the pitch analysis method requires a 60-millisecond analysis window, i.e., in order to measure the F0 at a time of, say, 0.850 seconds, Praat needs to consider a part of the sound that runs from 0.820 to 0.880 seconds. These 60 milliseconds correspond to 3 maximum pitch periods ($3/50 = 0.060$).”

The manual specifies:

$$\text{Analysis Window Length} = \frac{3}{\text{pitch_floor}} \quad (7)$$

For the group’s default pitch floor of 75 Hz, the analysis window is:

$$T_{\text{window}} = \frac{3}{75} = 0.040 \text{ s} = 40 \text{ ms} \quad (8)$$

Discrepancy Note on Slide 27: Slide 27 states: “*Pitch floor = 75Hz => $T = 1 / 75 = 0.04$ s (analysis window length)*”. Notice that $1/75 \approx 0.013333... \text{ s} \neq 0.04 \text{ s}$. The formula printed on Slide

27 contains a typographical error in the numerator (1/75 instead of 3/75), but the calculated duration 0.04 s is mathematically identical to the Praat manual’s 3-period identity $3/75 = 0.04$ s.

Furthermore, the Praat manual for **Sound: To Pitch...** defines the default time step (hop):

$$\Delta t = \frac{0.75}{\text{pitch_floor}} = \frac{0.75}{75} = 0.010 \text{ s} = 10 \text{ ms} \quad (9)$$

Reconstructing the 142-Point Count and Centering Offset. Slides 28–30 trace the arithmetic:

1. **Interior Span for Complete Windows (Slide 28):** For an analysis window of length 0.04 s to lie entirely within the audio bounds $[0, 1.450667 \text{ s}]$, the span available for window centers is:

$$T_{\text{interior}} = 1.450667 \text{ s} - 0.040000 \text{ s} = 1.410667 \text{ s} \quad (10)$$

2. **Number of Steps and Point Count (Slide 29):** With a time step of $\Delta t = 0.01$ s, the number of full steps is:

$$\text{Hops} = \left\lfloor \frac{1.410667}{0.01} \right\rfloor = 141 \implies \text{Total Points} = 141 + 1 = 142 \text{ points} \quad (11)$$

3. **Centering Offset (Slide 30):** The 141 hops span $141 \times 0.01 = 1.410000$ s, leaving a residual:

$$T_{\text{residual}} = 1.410667 \text{ s} - 1.410000 \text{ s} = 0.000667 \text{ s} \quad (12)$$

Praat’s centering mechanism splits this residual equally between the beginning and the end of the file:

$$\text{Centering Offset} = \frac{0.000667 \text{ s}}{2} = 0.0003335 \text{ s} \approx 0.00033 \text{ s} \quad (13)$$

The center of the first window is located at:

$$t_1 = \frac{T_{\text{window}}}{2} + \text{Centering Offset} = \frac{0.04}{2} + 0.00033333 = 0.02033333 \text{ s} \quad (14)$$

This matches the first timestamp extracted by the group on Slide 25 (0.02033333 s) to eight decimal places, verifying Praat’s internal framing cadence.

Meeting 23/09/2026: Rounding Discrepancy Budget and Edge Behavior (Slides 31–32)

The final presentation examines numerical edge effects under the title: “*ARISING ISSUES (THIS PART IS NOTED FOR PAPER WRITING; IN IMPLEMENTATION, PYTHON AUTOMATICALLY HANDLES TRUNCATION)*” (Slide 31).

Slide 32 considers a concrete scenario:

- Reference audio (Sarah): $T_1 = 1.42$ s, $n_1 = 139$ pitch points.
- Learner audio (User): $T_2 = 2.00$ s, with point count labeled as **n1 = 196** on the slide.

Typographical Note on Slide 32: The slide prints “ $T2 = 2s$, $n1 = 196$ ”, duplicating the variable name n_1 instead of labeling the User point count as $n_2 = 196$.

The slide then computes an effective rescaled temporal step:

$$\theta = \frac{T_1}{n_2} = \frac{1.42 \text{ s}}{196} = 0.0072448979 \dots \text{ s} \approx 0.007 \text{ s} \quad (15)$$

(Note: the slide prints the formula symbol as $\theta = \frac{T_2}{n_1} = 0.00724489s$, swapping variable indices in the formula while correctly evaluating $1.42/196$).

The Truncation Budget. Slide 32 examines what occurs if θ is rounded or truncated to three decimal places (0.007 s):

1. Reconstructed duration across 196 samples:

$$T_{\text{reconstructed}} = 196 \times 0.007 \text{ s} = 1.372 \text{ s} \quad (16)$$

2. Temporal discrepancy between Sarah’s original audio and the rescaled representation:

$$\Delta T = 1.420 \text{ s} - 1.372 \text{ s} = 0.048 \text{ s} \quad (17)$$

3. Discrepancy expressed in standard 0.01 s pitch frames:

$$\text{Missed Frames} = \frac{1.420 \text{ s} - 1.372 \text{ s}}{0.01 \text{ s}} = \frac{0.048 \text{ s}}{0.01 \text{ s}} = 4.8 \text{ points} \quad (18)$$

Slide 32 concludes: “*Misses approximately $(1.42 - 1.372)/0.01 = 4$ to 5 pitch points $\implies$ Thus approximating will miss or leave surplus points. We treat this as an allowable error tolerance.*”

Laboratory Recomputation for Duration 2.00 s. To cross-check the slide’s claim of 196 points for a 2.00 s audio under the verified 18/09 framing setup (pitch floor 75 Hz, window 0.04 s, hop 0.01 s):

$$T_{\text{interior}} = 2.000000 \text{ s} - 0.040000 \text{ s} = 1.960000 \text{ s} \quad (19)$$

The number of hops is:

$$\text{Hops} = \frac{1.960000}{0.01} = 196 \implies \text{Total Samples} = 196 + 1 = 197 \text{ points} \quad (20)$$

The slide’s count of 196 reflects taking $1.96/0.01 = 196$ directly without the fencepost +1, or omitting the terminal boundary point in the Python array slice.

1.6.2 What That Contract Already Buys

A critical responsibility of any incoming researcher joining an active project is to understand what the existing design already achieves before analyzing unhandled edge cases. A reader who only encounters open cases risks forming the mistaken impression that the pipeline was arbitrary or deficient. In reality, the contract constructed across the September 2026 slides establishes several profound engineering advantages:

1. **Minimal Computational Complexity (Slides 5, 11):** Evaluating linear interpolation (`np.interp`) and Equation (1) requires only 1D scalar accumulation, multiply-adds, and a single square root. On an embedded CPU, computing Pearson correlation across a 500-point array takes less than 10 μs . In hardware acceleration, this logic fits comfortably into a handful of LUTs and a single DSP slice, requiring negligible logic and zero off-chip memory bandwidth.
2. **Invariance to Vocal Register and Dynamic Range (Slides 6, 8):** By subtracting means $(\bar{x}, \bar{y})$ and dividing by standard deviations (σ_X, σ_Y) , the metric decouples evaluation from absolute pitch and amplitude. An adult male student with a natural pitch floor of 90 Hz can be compared directly against Sarah ($F_0 \approx 220$ Hz) without register bias or vocal strain penalties.
3. **Zero Training Pipeline and Zero Weight Footprint (Slides 2, 10):** Unlike neural acoustic models that demand gigabytes of training data, CTC loss alignments, and megabytes of on-chip weight storage, the Pearson sliding window operates entirely as a closed-form deterministic algorithm. It requires zero training epochs, zero parameter storage, and zero memory thrashing.
4. **Self-Contained Word Localization Without Forced Alignment (Slides 19–23):** Deploying an external forced aligner (such as a Kaldi or Montreal Forced Aligner pipeline) introduces heavy phonetic dictionaries, G2P converters, and acoustic hidden Markov models. The group’s analytical formula ($W = 2N/(K + 2)$ with $S = W/2$) provides an elegant geometric proxy that localizes scores along the utterance timeline using only the word count K .

5. **Empirically Verified Framing Infrastructure (Slides 24–30):** The team conducted rigorous signal-level validation of Praat’s framing mechanics, verifying the analysis window (40 ms), the hop cadence (10 ms), and the sub-millisecond boundary centering (0.00033 s). This empirical discipline provides a solid foundation for any subsequent signal-processing expansion.

1.6.3 Open Cases in the Evaluation Pipeline

Having identified the core strengths of the contract, we turn to the open cases where physical speech dynamics diverge from the operational assumptions of the slides.

Independence of the Open Cases. It is vital to recognize that the four open cases detailed below are **mutually independent**. They do not stem from a single shared bug; rather, each case addresses an independent dimension of the speech signal. Resolving unvoiced frames (Case 3) does not resolve non-linear tempo variations (Case 2); localizing acoustic word edges (Case 4) does not verify whether the learner pronounced the correct vowel formants (Case 1). Each case must be evaluated on its own mathematical merits.

In this section, we do not propose a dynamic time warping (DTW) solution or pre-empt the team’s architectural choices; we strictly document the assumption, the uncovered physical reality, the consequence of the current rule, the meeting question, and assign one of two engineering labels: **patch this line** or **replace this line**.

Open Case 1: The Measure is Pitch Trend, Not Phonetic Identity

1. **Assumption Used on the Slides (Slides 2, 5):** The pipeline assumes that evaluating Pearson correlation on fundamental frequency (F_0) over time is sufficient to verify whether a learner read the prompt correctly (*“checking whether the user reads like the model”*, Slide 2).
2. **Physical Case Not Covered by This Assumption:** Human speech acoustics decouple the vocal source (fundamental frequency F_0 , determined by vocal fold vibration) from the vocal tract filter (formant resonances $F_1, F_2, F_3, \dots$, determined by the tongue, jaw, and lip geometry). Pearson correlation on pitch evaluates only whether the speaker’s vocal folds tensed and relaxed in directional harmony with the reference. It is completely blind to spectral formant envelopes.

Consequently, this assumption fails to cover two common student behaviors:

- *Humming Exploits:* A student can keep their mouth entirely closed and hum the rising and falling melody of the sentence without articulating a single consonant or vowel.
 - *Phonetic Substitutions with Matching Intonation:* A student reciting an entirely incorrect sentence (for example, saying *“I hate that dog”* instead of *“I love this cat”*) with the same declarative pitch trajectory will produce a nearly identical pitch sequence X .
3. **What the Current Rule Would Do:** Because the pitch trajectory matches Sarah’s intonation trend, the numerator $\sum(x_i - \bar{x})(y_i - \bar{y})$ remains strongly positive, yielding $r \geq 0.3$. The current rule assigns a passing score to hummed audio or completely incorrect words.
 4. **Meeting Question for the Group:** Does the research group intend for pitch Pearson correlation to serve as a complete standalone pronunciation scoring engine, or is it envisioned strictly as an auxiliary prosody score layered on top of an independent acoustic phonetic recognizer?
 5. **Engineering Label:** **replace this line.** Pitch correlation cannot verify phonetic identity; evaluating whether the learner articulated the prompt requires an acoustic spectral metric (such as spectral distance or phonetic log-likelihoods).

Open Case 2: Equal Index After Linear Resize Is Not the Same Place in the Sentence

1. **Assumption Used on the Slides (Slide 11):** The pipeline assumes that global linear interpolation (`np.interp`) mapping the learner’s pitch array to match the length of Sarah’s pitch array aligns corresponding phonetic moments across the sentence.
2. **Physical Case Not Covered by This Assumption:** Human speech rate is fundamentally non-uniform. When a non-native learner hesitates, prolongs a challenging vowel, or pauses before an unfamiliar word, the temporal expansion is concentrated entirely on that specific phoneme, while the remainder of the sentence may be spoken at normal speed.

For example, consider a learner saying “*Gooooo... morning*” (1.8 s) where the vowel /u/ in “Good” is prolonged across 60% of the utterance, compared to an instructor saying “*Good morning*” (1.0 s) where “Good” occupies only 35% of the utterance.

3. **What the Current Rule Would Do:** Global linear interpolation stretches or compresses all intervals uniformly. Because the learner’s first word was elongated, uniform downsampling forces the tail of the learner’s first word into the temporal slot occupied by Sarah’s second word.

At normalized frame $k = 45$, the learner is still concluding the rising intonation of “Good” ($\Delta x > 0$), while Sarah is already descending on the unaccented syllable of “morning” ($\Delta y < 0$). Multiplying these opposing slopes yields:

$$(x_{45} - \bar{x})(y_{45} - \bar{y}) < 0 \quad (21)$$

This phase inversion forces the covariance negative ($r < 0$). Even though the student pronounced both words with excellent native-like intonation, the rigid uniform ruler produces a catastrophic false negative failure.

4. **Meeting Question for the Group:** How should the pipeline decouple local speaking rate variations from prosodic evaluation? Should the alignment mechanism allow non-linear temporal elasticity to absorb vowel stretching before computing intonation correlation?
5. **Engineering Label: replace this line.** Uniform linear interpolation is structurally incapable of mapping non-linearly distorted acoustic timelines onto one another.

Open Case 3: The Numeric State of Unvoiced Frames

1. **Assumption Used on the Slides (Slide 5):** Equation (1) assumes that $X = \{x_1, \dots, x_n\}$ and $Y = \{y_1, \dots, y_n\}$ are dense, continuous, well-defined real-valued vectors across all frames $i \in \{1, \dots, n\}$.
2. **Physical Case Not Covered by This Assumption:** Speech is not continuously periodic. Consonants such as unvoiced fricatives (/s/, /sh/, /f/), unvoiced plosive closures and bursts (/p/, /t/, /k/), and inter-word pauses involve no vocal fold vibration. During these intervals, fundamental frequency F_0 is physically non-existent.
3. **What the Current Rule Would Do:** The slide deck does not state how the group’s Python baseline treats unvoiced frames. Opening the Praat manual indicates that Praat flags unvoiced frames as undefined. In numerical implementations, this leads to three unhandled failure modes:
 - *If unvoiced frames are stored as NaN:* Any sum $\sum(x_i - \bar{x})$ containing a NaN propagates across all accumulators, resulting in $r = \text{NaN}$ and crashing the downstream comparator.

- *If unvoiced frames are filled with 0.0 Hz:* A jump from 0.0 Hz to a voiced pitch of 150 Hz represents an artificial delta of 150 Hz. These massive artificial discontinuities completely dominate the variance sums $\sum(x_i - \bar{x})^2$, distorting the Pearson correlation into a measure of silence overlap rather than intonation trend.
 - *If unvoiced frames are deleted:* Deleting unvoiced frames destroys the uniform 0.01 s temporal grid, making linear interpolation geometrically invalid.
4. **Meeting Question for the Group:** What is the formal specification for unvoiced frames in the group’s Python code? Are unvoiced intervals masked with a boolean voiced/unvoiced flag, linearly interpolated through, or evaluated strictly on mutually voiced frame pairs?
 5. **Engineering Label: patch this line.** The Pearson correlation framework can be retained by augmenting the contract with an explicit mutually voiced frame mask $\mathcal{V} = \{i \mid X_i > 0 \wedge Y_i > 0\}$ and enforcing a minimum voiced frame ratio threshold.

Open Case 4: Window Staining Cascades and the Rounding Residual

1. **Assumption Used on the Slides (Slides 20–23, 32):** The sliding window formula $W = 2N/(K + 2)$ with 50% stride ($S = W/2$) accurately isolates word-level mistakes, and the 4-to-5 frame discrepancy identified on Slide 32 is negligible allowable error (*“allowable error tolerance”*).
2. **Physical Case Not Covered by This Assumption:** Geometric windows sized purely by total sentence length N and word count K assume that all words have identical durations. In real language, English words vary wildly in duration: monosyllabic function words (“a”, “in”, “the”) may last only 80 ms, while polysyllabic content words (“pronunciation”, “extraordinary”) span 600 ms or more.

Consequently, fixed geometric windows do not align with acoustic word boundaries. An intonation error occurring strictly within Word 2 will spill across both Window 2 and Window 3.

3. **What the Current Rule Would Do:** Under the Group Condemnation Rule (Slide 23):

“If any window has $r < 0.3$, then all words falling on that window are marked incorrect, even if they also fall on another window with $r \geq 0.3$.”

Because Window 2 drops below 0.3, Word 1 and Word 2 are marked incorrect. Because Window 3 also drops below 0.3, Word 2 and Word 3 are marked incorrect. A single localized prosodic error on Word 2 thus stains and fails all three words in the sentence (Figure 2).

Furthermore, the rounding discrepancy of 4 to 5 frames (40 to 50 ms) documented on Slide 32 represents a significant portion of an 80 ms function word, shifting window edges across phoneme boundaries unpredictably.

4. **Meeting Question for the Group:** Should word boundaries be derived from acoustic forced alignment rather than synthetic geometric formulas, and should the binary condemnation rule be relaxed to a weighted voting scheme based on the percentage of word overlap?
5. **Engineering Label: patch this line.** The underlying pitch Pearson comparison can be preserved by replacing the synthetic window geometry with acoustic word boundary timestamps and replacing the binary condemnation rule with fractional credit.

1.6.4 Stages the Report Does Not Contain, and Four Unchosen Candidates

Flow Stages Not Contained in the Report. Reviewing the five-stage architecture established in Section 1.6.0 confirms that the group report focuses specifically on Stage 4 and an initial rule in Stage 5. The pipeline does not contain the following operational stages:

1. **Stage 1: Upstream Audio Conditioning and Dynamic VAD.** The slides operate on a pre-trimmed static file (`sarah_trimmed.wav`, Slide 25). The report does not contain real-time Voice Activity Detection to crop user pauses, DC bias filters, or automated energy normalization to handle low-cost USB edge microphones.
2. **Stage 2: Acoustic Forced Alignment.** The report does not include a phone- or word-level aligner (such as an HMM-GMM Viterbi decoder or CTC forced aligner). Word segmentation is performed entirely geometrically via the $K + 1$ window formula ($W = 2N/(K + 2)$).
3. **Stage 3: Spectral Phoneme Quality Verification.** The report does not evaluate vocal tract filter properties (formants F_1, F_2, F_3) or acoustic model posteriors. Spectral phone correctness is assumed to correlate with fundamental frequency trajectory.
4. **Stage 5: Composite Pedagogical Rubric Calibration.** While the report establishes the $r < 0.3$ threshold on Slide 23, it does not define how prosodic correlation combines with phonetic accuracy, lexical stress, or speaking rate to produce a standardized educational score (e.g., CEFR or IELTS scales).

Four Architecture Candidates Under Uniform Evaluation Axes

To provide the research group with actionable engineering pathways without pre-judging the outcome, we evaluate four distinct architectural candidates. Each candidate is examined across the exact same four analytical axes:

- **Axis 1 (Preserved Group Idea):** The percentage and components of the September 2026 contract retained.
- **Axis 2 (Open Cases Covered):** Which of the four independent open cases (Section 1.6.3) the candidate resolves.
- **Axis 3 (Output Granularity):** The resolution of feedback delivered to the student (sentence-level, window-level, word-level, or phoneme-level).
- **Axis 4 (Compute Cost on CPU and Hardware Profile):** The algorithmic complexity and CPU execution time, accompanied by a concise one-phrase characterization of FPGA mapping.

Candidate A: Keep the Contract.

- *Axis 1 (Preserved Group Idea):* 100%. Retains global linear interpolation (`np.interp`), the $K + 1$ sliding window geometry ($W = 2N/(K + 2)$), Praat 0.01 s framing, and the Group Condemnation Rule ($r < 0.3$).
- *Axis 2 (Open Cases Covered):* Covers none of the four open cases. The pipeline treats the 4 to 5 frame rounding discrepancy documented on Slide 32 as an allowable engineering tolerance (“allowable error tolerance”).
- *Axis 3 (Output Granularity):* Coarse window-level pass/fail status mapped to overlapping word groups.

- *Axis 4 (Compute Cost)*: Minimal on CPU. Composed entirely of $\mathcal{O}(N)$ 1D scalar interpolations and dot products, executing in under 10 μ s per sentence. On FPGA, it synthesizes into minimal fabric logic and a single DSP block.

Candidate B: Add Word Edges, Then Pearson Per Word.

- *Axis 1 (Preserved Group Idea)*: High ($\approx 75\%$). Retains Praat pitch extraction, Pearson intonation correlation, and comparative scoring against Sarah. Replaces the synthetic geometric windows ($W = 2N/(K + 2)$) with acoustic word boundaries obtained from an upstream aligner.
- *Axis 2 (Open Cases Covered)*: Resolves Open Case 4 (Window Staining Cascades) completely: each word is evaluated strictly within its true acoustic boundaries $[t_{\text{start}}, t_{\text{end}}]$, eliminating cross-word staining. Partially mitigates Open Case 2 at word boundaries, but does not resolve sub-word non-uniform vowel stretching. Leaves Open Case 1 (phone identity) and Case 3 (unvoiced handling) open.
- *Axis 3 (Output Granularity)*: True word-level score (K independent numerical correlation scores).
- *Axis 4 (Compute Cost)*: Moderate on CPU. Dominated by the upstream forced alignment pass (10–50 ms depending on model size); once word boundaries are extracted, computing K independent Pearson correlations takes $< 50 \mu$ s. On FPGA, requires word timestamp metadata streamed from a host processor.

Candidate C: Replace Only the Resize Line (Elastic Align, Then Pearson on Path).

- *Axis 1 (Preserved Group Idea)*: Moderate ($\approx 50\%$). Retains the core philosophy of comparative prosodic scoring against Sarah and the Pearson intonation metric. Replaces only the global linear interpolation line (`np.interp`) with non-linear dynamic time warping along an elastic path $\mathcal{P}$.
- *Algorithmic Formulation*: For learner sequence X of length N and reference sequence Y of length M , local cost is computed as Euclidean distance $d(i, j) = \|x_i - y_j\|$. The accumulated cost grid satisfies the standard dynamic programming recurrence:

$$D(i, j) = d(i, j) + \min(D(i - 1, j), D(i, j - 1), D(i - 1, j - 1)) \quad (22)$$

Tracing the optimal warping path $\mathcal{P} = ((i_1, j_1), \dots, (i_L, j_L))$ from $(1, 1)$ to (N, M) elastically aligns corresponding acoustic events. Pearson intonation correlation is subsequently evaluated strictly along the reindexed warping path $\mathcal{P}$.

- *Axis 2 (Open Cases Covered)*: Resolves Open Case 2 (Equal Index After Resize) by elastically absorbing local vowel stretching and pauses, completely preventing false negative phase inversions ($r < 0$). When paired with a mutually voiced frame mask, it isolates Open Case 3. However, it does not resolve Open Case 1 unless multi-dimensional spectral features are incorporated into local distance $d(i, j)$.
- *Axis 3 (Output Granularity)*: Frame-level alignment path mapped into localized window or word scores.
- *Axis 4 (Compute Cost)*: Moderate on CPU. Evaluating the $N \times M$ grid for a 5-second sentence ($500 \times 500 = 250,000$ cells) requires approximately 5–15 ms in Python. On FPGA, it maps efficiently to a streaming systolic processing array.

Candidate D: Replace the Measure (Goodness of Pronunciation or Acoustic Embeddings).

- *Axis 1 (Preserved Group Idea)*: Low ($\approx 20\%$). Retains only the overarching goal of automated English pronunciation assessment. Replaces pitch Pearson correlation with acoustic model posterior probabilities (Goodness of Pronunciation, GOP) or self-supervised speech representation embeddings (e.g., wav2vec 2.0 or Conformer representations).
- *Axis 2 (Open Cases Covered)*: Resolves Open Case 1 (Phone Correctness) directly by evaluating spectral acoustic log-likelihoods, resolves Open Case 3 natively (neural acoustic models handle voiced and unvoiced frames inherently), and provides intrinsic phonetic forced alignment.
- *Axis 3 (Output Granularity)*: Fine-grained phoneme-level, syllable-level, and word-level posterior scores.
- *Axis 4 (Compute Cost)*: High on CPU. Demands full neural network forward inference, multi-megabyte parameter storage, and intensive matrix-vector multiplications, taking 50–200 ms on an embedded CPU. On FPGA, it requires a dedicated deep learning neural network accelerator core and an extensive ongoing engineering burden to train, prune, and maintain the acoustic model.

Explicit Neutrality Note: We emphasize that Candidate C is not declared the winner, nor is Candidate A rejected as incorrect. If the project’s computational budget is severely restricted and learners are instructed to match a fixed metronome cadence, Candidate A provides an exceptionally lightweight baseline. If phoneme verification is paramount, Candidate D is mandatory despite its substantial model footprint. The choice between these four candidates belongs entirely to the research group.

1.6.5 The Group Meeting Decision Page

To facilitate a productive discussion in the upcoming research group meeting, Table 1 synthesizes the four architecture candidates. In accordance with the objective tone of this review, the final column is intentionally left blank for the team’s collective decision.

Table 1: **Architectural Decision Matrix for Read-Aloud Speech Evaluation.** Summary of trade-offs across the four candidates. The final column is left entirely open for the research group to select the operational path.

Candidate	Preserved Group Idea	Open Cases Covered	Output Grain	CPU Latency	FPGA Profile	Engineering Burden	Group Chooses
A: Keep Contract	100% (linear re-size, sliding windows, $r < 0.3$ rule).	None (treats 4–5 frame residual as allowable error).	Coarse window pass/fail.	$< 10 \mu s$	Minimal LUTs, single DSP block.	Zero new software; code is complete.	
B: Add Word Edges	$\approx 75\%$ (pitch Pearson and Sarah ruler survive).	Case 4 (eliminates window staining).	Word-level (K scalar scores).	10–50 ms (aligner bound)	Word timestamps streamed from host.	Integrate forced aligner (MFA/Kaldi) or VAD segmentation.	
C: Elastic Alignment	$\approx 50\%$ (pitch Pearson on path $\mathcal{P}$ survives).	Case 2 (absorbs vowel tempo), isolates Case 3.	Warped path $\rightarrow$ word/window.	5–15 ms (2.5×10^5 cells)	Streaming systolic PE array.	Implement 2D DP matrix recurrence; define local metric.	
D: Replace Measure	$\approx 20\%$ (only high-level goal survives).	Case 1 (phones), Case 3 (unvoiced), intrinsic alignment.	Fine-grained phoneme posteriors.	50–200 ms (neural forward)	Dedicated deep learning NPU accelerator.	Train, prune, and quantize neural acoustic model (GOP/Conformer).	

Actionable Meeting Questions for the Group

The decision matrix reflects four concrete architectural crossroads. We submit the following questions, drawn directly from the open cases analyzed in Section 1.6.3, to guide the team’s agenda:

1. **On Phonetic Scope (Open Case 1):** Is the current pipeline expected to detect unpronounced or substituted words independently, or is the edge voice system designed as a two-tier device where an upstream phoneme recognizer verifies phonetic correctness before passing pitch vectors to this stage?
2. **On Temporal Alignment (Open Case 2):** Does the group wish to preserve global linear interpolation by instructing students to adhere to a rigid metronome pace (Candidate A), segment utterances by word boundaries (Candidate B), or adopt elastic non-linear dynamic time warping (Candidate C) to accommodate natural vowel prolongation?
3. **On Unvoiced Frame Specification (Open Case 3):** What precise mathematical rule will be committed to the Python codebase for unvoiced frames ($F_0 = 0$ or undefined)? Should the calculation adopt a mutually voiced frame mask, linear pitch interpolation across voiceless gaps, or an explicit voiceless frame penalty?
4. **On Rubric Calibration (Open Case 4):** How should the team refine the Group Condemnation Rule? Should the binary fail condition ($r < 0.3$ condemns all overlapping words) be replaced by a proportional credit rule based on the percentage of overlap between word acoustic boundaries and sliding windows?